

Introducción a la Programación con Python

Semana 6: Entrada/Salida de Archivos

▽Nablabs

Contenido

- 1 ¿Por qué trabajar con archivos?
- 2 Función open()
- 3 Modos de apertura
- 4 Lectura de archivos
- 5 Escritura en archivos
- 6 Bloque with
- 7 Archivos CSV
- 8 Rutas de archivos
- 9 Manejo de errores
- 10 Proyecto integrador

¿Para qué usar archivos?

-  **Persistencia:** Los datos sobreviven al cierre del programa
- Intercambio:** Compartir datos entre programas
-  **Almacenamiento:** Guardar grandes cantidades de información
-  **Registro:** Logs y historial de actividades

¡Los archivos son fundamentales!

¿Cómo funciona?

Abre un archivo y retorna un objeto de archivo para trabajar con él

Sintaxis básica:

```
1 archivo = open("nombre.txt", "modo")
2 # ... trabajar con el archivo ...
3 archivo.close() # Siempre cerrar!
```

 Siempre debes cerrar el archivo con close()

Modos principales

-  **'r'** - **Lectura** (read): Solo leer, archivo debe existir
- 'w'** - **Escritura** (write): Crea o sobrescribe archivo
- +** **'a'** - **Añadir** (append): Agrega al final del archivo
-  **'r+'** - **Lectura/Escritura**: Lee y escribe, archivo debe existir
-  **'x'** - **Creación exclusiva**: Error si el archivo existe

 Modo por defecto: 'r'

Modos adicionales

 't' - **Texto** (por defecto): Archivo de texto

 'b' - **Binario**: Para archivos binarios (imágenes, etc.)

Ejemplos de combinaciones:

```
1 # Lectura de texto (modo por defecto)
2 archivo1 = open("datos.txt", "r")
3
4 # Escritura binaria
5 archivo2 = open("imagen.png", "wb")
6
7 # Lectura y escritura de texto
8 archivo3 = open("config.txt", "r+")
```

Método read()

¿Qué hace?

Lee todo el contenido del archivo como una cadena

Supongamos que `saludo.txt` contiene:

```
Hola mundo  
Bienvenido a Python  
¡Aprender es divertido!
```

```
1 archivo = open("saludo.txt", "r")  
2 contenido = archivo.read()  
3 print(contenido)  
4 archivo.close()
```

➔ Salida

```
Hola mundo  
Bienvenido a Python  
¡Aprender es divertido!
```

Método readline() ≡

¿Qué hace?

Lee una línea a la vez del archivo

```
1 archivo = open("saludo.txt", "r")
2
3 linea1 = archivo.readline()
4 linea2 = archivo.readline()
5
6 print(f"Primera: {linea1.strip()}")
7 print(f"Segunda: {linea2.strip()}")
8
9 archivo.close()
```

➔ Salida

Primera: Hola mundo

Segunda: Bienvenido a Python

💡 `strip()` elimina espacios y saltos de línea

Método readlines()

¿Qué hace?

Lee todas las líneas y las retorna como una lista

```
1 archivo = open("saludo.txt", "r")
2 lineas = archivo.readlines()
3 archivo.close()
4
5 print(f"Total de líneas: {len(lineas)}")
6
7 for i, linea in enumerate(lineas, 1):
8     print(f"{i}: {linea.strip()}")
```

➔ Salida

Total de líneas: 3

1: Hola mundo

2: Bienvenido a Python

3: ¡Aprender es divertido!

Iterar sobre un archivo

Forma más eficiente

Puedes iterar directamente sobre el objeto archivo

```
1 archivo = open("saludo.txt", "r")
2
3 for linea in archivo:
4     print(f">>> {linea.strip()}")
5
6 archivo.close()
```

➔ Salida

```
>>> Hola mundo
>>> Bienvenido a Python
>>> ¡Aprender es divertido!
```

✔ Más eficiente en memoria, procesa línea por línea

Método write()

¿Qué hace?

Escribe una cadena en el archivo

```
1 archivo = open("mensaje.txt", "w")
2
3 archivo.write("Primera línea\n")
4 archivo.write("Segunda línea\n")
5 archivo.write("Tercera línea")
6
7 archivo.close()
8
9 # Verificar el contenido
10 archivo = open("mensaje.txt", "r")
11 print(archivo.read())
12 archivo.close()
```

➔ Contenido de mensaje.txt

Primera línea
Segunda línea

Método writelines()

¿Qué hace?

Escribe una lista de cadenas en el archivo

```
1 frutas = ["Manzana\n", "Banana\n", "Cereza\n"]
2
3 archivo = open("frutas.txt", "w")
4 archivo.writelines(frutas)
5 archivo.close()
6
7 # Leer el resultado
8 archivo = open("frutas.txt", "r")
9 print(archivo.read())
10 archivo.close()
```

➔ Contenido de frutas.txt

Manzana
Banana
Cereza

Modo append 'a' +

¿Qué hace?

Agrega contenido al final sin borrar lo existente

```
1 # Crear archivo inicial
2 archivo = open("log.txt", "w")
3 archivo.write("Entrada 1\n")
4 archivo.close()
5
6 # Agregar más contenido
7 archivo = open("log.txt", "a")
8 archivo.write("Entrada 2\n")
9 archivo.write("Entrada 3\n")
10 archivo.close()
11
12 # Verificar
13 archivo = open("log.txt", "r")
14 print(archivo.read())
15 archivo.close()
```

Bloque with

¿Por qué usar with?

Cierra el archivo automáticamente, incluso si hay errores

```
1 # Sin with (no recomendado)
2 archivo = open("datos.txt", "r")
3 contenido = archivo.read()
4 archivo.close() # Puedes olvidar esto
5
6 # Con with (recomendado)
7 with open("datos.txt", "r") as archivo:
8     contenido = archivo.read()
9 # El archivo se cierra automáticamente aquí
10
11 print(contenido)
```

✓ ¡Mejor práctica! Más seguro y limpio

with: Múltiples operaciones </>

```
1 # Lectura con with
2 with open("entrada.txt", "r") as archivo:
3     lineas = archivo.readlines()
4
5 # Escritura con with
6 with open("salida.txt", "w") as archivo:
7     for i, linea in enumerate(lineas, 1):
8         archivo.write(f"{i}. {linea}")
9
10 # Verificar resultado
11 with open("salida.txt", "r") as archivo:
12     print(archivo.read())
```

➔ Salida (ejemplo)

1. Primera línea
2. Segunda línea
3. Tercera línea

¿Sabías que?

Puedes abrir varios archivos a la vez en un solo with

```
1 # Copiar contenido de un archivo a otro
2 with open("origen.txt", "r") as origen, \
3     open("destino.txt", "w") as destino:
4
5     contenido = origen.read()
6     destino.write(contenido)
7
8 print("Archivo copiado exitosamente")
```

 Ambos archivos se cierran automáticamente



¿Qué es CSV?

CSV = Comma-Separated Values (Valores Separados por Comas)
Formato simple para almacenar datos tabulares

Ejemplo de CSV:

```
nombre,edad,ciudad  
Ana,25,Madrid  
Juan,30,Barcelona  
María,28,Valencia
```

 Común en hojas de cálculo y bases de datos

CSV sin módulo: Lectura

```
1 with open("datos.csv", "r") as archivo:
2     for linea in archivo:
3         # Separar por comas
4         campos = linea.strip().split(",")
5         print(campos)
```

➔ Salida

```
['nombre', 'edad', 'ciudad']
Ana', '25', 'Madrid'

Juan', '30', 'Barcelona'
```

 Funciona, pero no maneja casos complejos

¿Por qué usar el módulo csv?

Maneja correctamente comillas, comas dentro de campos, etc.

```
1 import csv
2
3 with open("datos.csv", "r") as archivo:
4     lector = csv.reader(archivo)
5     encabezados = next(lector)
6     print(f"Columnas: {encabezados}")
7
8     for fila in lector:
9         nombre, edad, ciudad = fila
10        print(f"{nombre}, {edad} años, {ciudad}")
```

➔ Salida

Columnas: ['nombre', 'edad', 'ciudad']

Ana, 25 años, Madrid

Juan, 30 años, Barcelona

¿Qué hace?

Lee cada fila como un diccionario usando los encabezados como claves

```
1 import csv
2
3 with open("datos.csv", "r") as archivo:
4     lector = csv.DictReader(archivo)
5
6     for fila in lector:
7         print(f"Nombre: {fila['nombre']}")
8         print(f"Edad: {fila['edad']}")
9         print(f"Ciudad: {fila['ciudad']}")
10        print("---")
```

➡ Salida

```
Nombre: Ana
Edad: 25
Ciudad: Madrid
---
```

```
1 import csv
2
3 # Datos a escribir
4 datos = [
5     ["nombre", "edad", "ciudad"],
6     ["Pedro", 35, "Sevilla"],
7     ["Laura", 22, "Bilbao"],
8     ["Carlos", 40, "Málaga"]
9 ]
10
11 with open("nuevos.csv", "w", newline='') as archivo:
12     escritor = csv.writer(archivo)
13     escritor.writerows(datos)
14
15 print("CSV creado exitosamente")
```

i `newline=''` previene líneas en blanco extra

```
1 import csv
2
3 personas = [
4     {"nombre": "Pedro", "edad": 35, "ciudad": "Sevilla"},
5     {"nombre": "Laura", "edad": 22, "ciudad": "Bilbao"},
6     {"nombre": "Carlos", "edad": 40, "ciudad": "Málaga"}
7 ]
8
9 with open("personas.csv", "w", newline='') as archivo:
10     campos = ["nombre", "edad", "ciudad"]
11     escritor = csv.DictWriter(archivo, fieldnames=campos)
12
13     escritor.writeheader() # Escribir encabezados
14     escritor.writerows(personas)
15
16 print("CSV con diccionarios creado")
```

💡 Útil cuando trabajas con diccionarios

Tipos de rutas

Absoluta: Ruta completa desde la raíz



Relativa: Ruta desde el directorio actual

```
1 # Ruta relativa (mismo directorio)
2 archivo1 = open("datos.txt", "r")
3
4 # Ruta relativa (subdirectorio)
5 archivo2 = open("archivos/datos.txt", "r")
6
7 # Ruta absoluta (Linux/Mac)
8 archivo3 = open("/home/usuario/datos.txt", "r")
9
10 # Ruta absoluta (Windows)
11 archivo4 = open("C:\\Users\\usuario\\datos.txt", "r")
```

 En Windows usa \\

Trabajo con rutas

El módulo `os.path` facilita el manejo de rutas multiplataforma

```
1 import os
2
3 # Verificar si existe
4 existe = os.path.exists("datos.txt")
5 print(f"Existe: {existe}")
6
7 # Es archivo o directorio?
8 es_archivo = os.path.isfile("datos.txt")
9 es_carpeta = os.path.isdir("archivos")
10
11 # Tamaño del archivo (en bytes)
12 tamaño = os.path.getsize("datos.txt")
13 print(f"Tamaño: {tamaño} bytes")
```


os.path: Nombres y extensiones

```
1 import os
2
3 # Nombre base del archivo
4 ruta = "/home/usuario/documentos/archivo.txt"
5 nombre = os.path.basename(ruta)
6 print(f"Nombre: {nombre}")    # "archivo.txt"
7
8 # Directorio
9 directorio = os.path.dirname(ruta)
10 print(f"Directorio: {directorio}")
11
12 # Separar nombre y extensión
13 nombre_base, extension = os.path.splitext(nombre)
14 print(f"Base: {nombre_base}")  # "archivo"
15 print(f"Extensión: {extension}")  # ".txt"
```

Construir rutas con `os.path.join`

¿Por qué usar `join`?

Construye rutas correctamente según el sistema operativo

```
1 import os
2
3 # Construir ruta multiplataforma
4 carpeta = "documentos"
5 subcarpeta = "proyectos"
6 archivo = "reporte.txt"
7
8 ruta = os.path.join(carpeta, subcarpeta, archivo)
9 print(f"Ruta: {ruta}")
10
11 # Usar la ruta
12 with open(ruta, "w") as f:
13     f.write("Contenido del reporte")
```

➔ Salida

Ruta: documentos/proyectos/reporte.txt

Errores comunes con archivos !

```
1 archivo_nombre = "inexistente.txt"
2
3 try:
4     with open(archivo_nombre, "r") as archivo:
5         contenido = archivo.read()
6         print(contenido)
7
8 except FileNotFoundError:
9     print(f"Error: '{archivo_nombre}' no existe")
10
11 except PermissionError:
12     print(f"Error: Sin permisos")
13
14 except Exception as e:
15     print(f"Error inesperado: {e}")
```

➔ Salida

Error: 'inexistente.txt' no existe

Buena práctica

Verificar la existencia del archivo antes de intentar abrirlo

```
1 import os
2
3 archivo_nombre = "datos.txt"
4
5 if os.path.exists(archivo_nombre):
6     with open(archivo_nombre, "r") as archivo:
7         print(archivo.read())
8 else:
9     print(f"Advertencia: '{archivo_nombre}' no existe")
10    # Crear el archivo o tomar otra acción
11    with open(archivo_nombre, "w") as archivo:
12        archivo.write("Archivo nuevo creado")
13    print("Archivo creado")
```

 Combina verificación con manejo de excepciones

Archivo: contactos.py

```
1 import csv
2 import os
3
4 ARCHIVO_CSV = "contactos.csv"
5
6 def crear_archivo_si_no_existe():
7     """Crea el archivo CSV si no existe"""
8     if not os.path.exists(ARCHIVO_CSV):
9         with open(ARCHIVO_CSV, "w", newline='') as f:
10             escritor = csv.writer(f)
11             escritor.writerow(
12                 ["nombre", "telefono", "email"])
```

Proyecto: Agregar contacto

```
1 def agregar_contacto(nombre, telefono, email):
2     """Agrega un nuevo contacto al archivo CSV"""
3     with open(ARCHIVO_CSV, "a", newline='') as f:
4         escritor = csv.writer(f)
5         escritor.writerow([nombre, telefono, email])
6     print(f"Contacto '{nombre}' agregado")
```

Proyecto: Listar contactos

```
1 def listar_contactos():
2     """Muestra todos los contactos"""
3     with open(ARCHIVO_CSV, "r") as f:
4         lector = csv.DictReader(f)
5         print("\n=== LISTA DE CONTACTOS ===")
6         for i, contacto in enumerate(lector, 1):
7             print(f"{i}. {contacto['nombre']}")
8             print(f"    Tel: {contacto['telefono']}")
9             print(f"    Email: {contacto['email']}")
10            print("---")
```

Proyecto: Buscar contacto 🔍

```
1 def buscar_contacto(nombre_buscar):
2     """Busca un contacto por nombre"""
3     encontrado = False
4     with open(ARCHIVO_CSV, "r") as f:
5         lector = csv.DictReader(f)
6         for contacto in lector:
7             if nombre_buscar.lower() in \
8                 contacto['nombre'].lower():
9                 print(f"\n{contacto['nombre']}")
10                print(f"Tel: {contacto['telefono']}")
11                print(f"Email: {contacto['email']}")
12                encontrado = True
13
14 if not encontrado:
15     print(f"No encontrado: '{nombre_buscar}'")
```


Proyecto: Menú principal

```
1 def menu():
2     crear_archivo_si_no_existe()
3
4     while True:
5         print("\n=== AGENDA DE CONTACTOS ===")
6         print("1. Agregar contacto")
7         print("2. Listar contactos")
8         print("3. Buscar contacto")
9         print("4. Salir")
10
11         opcion = input("\nOpción: ")
12
13         if opcion == "1":
14             nombre = input("Nombre: ")
15             telefono = input("Teléfono: ")
16             email = input("Email: ")
17             agregar_contacto(nombre, telefono, email)
```



```
1 elif opcion == "2":
2     listar_contactos()
3
4 elif opcion == "3":
5     nombre = input("Buscar nombre: ")
6     buscar_contacto(nombre)
7
8 elif opcion == "4":
9     print(" Hasta luego!")
10    break
11 else:
12     print("Opción inválida")
13
14 if __name__ == "__main__":
15     menu()
```

Proyecto alternativo: Analizador CSV

```
1 import csv
2 import statistics
3
4 def analizar_ventas(archivo):
5     """Analiza un archivo CSV de ventas"""
6     with open(archivo, "r") as f:
7         lector = csv.DictReader(f)
8         ventas = []
9         for fila in lector:
10             ventas.append(float(fila['monto']))
11
12         print(f"Total: ${sum(ventas):.2f}")
13         print(f"Promedio: ${statistics.mean(ventas):.2f}")
14     )
15
16     print(f"Máxima: ${max(ventas):.2f}")
17     print(f"Mínima: ${min(ventas):.2f}")
```

¿Qué aprendimos hoy?

- ✓ Función `open()` y modos de apertura
- ✓ Lectura: `read()`, `readline()`, `readlines()`
- ✓ Escritura: `write()`, `writelines()`
- ✓ Bloque `with` para manejo seguro
- ✓ Archivos CSV con el módulo `csv`
- ✓ Rutas de archivos con `os.path`
- ✓ Manejo de errores al trabajar con archivos
- ✓ Proyectos prácticos: Agenda y análisis de datos

¡Los archivos son esenciales en programación!



¡Preguntas!

Gracias por su atención